

Министерство науки и высшего образования Российской Федерации
**Федеральное государственное бюджетное образовательное учреждение высшего
образования**
«Владимирский государственный университет имени Александра Григорьевича и Николая
Григорьевича Столетовых»
(ВлГУ)

Кафедра информационных систем и программной инженерии

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА
к курсовому проекту по дисциплине
«Распределённые программные системы»
на тему

Проектирование и разработка информационной системы
«База отдыха Re-Center»

Выполнили:

студенты гр. ИСТ-123:

Б. С. Турков,

Г. С. Мошталёв

Приняла:

доц. каф. ИСПИ

Г. В. Проскурина

АННОТАЦИЯ

Пояснительная записка содержит описание проектирования и разработки информационной системы «База отдыха Re-Center» — распределённой веб-системы для автоматизации бизнес-процессов туристического комплекса.

В документе представлены: анализ предметной области, описание пяти основных прецедентов использования, функциональные и нефункциональные требования, архитектура системы, модель данных с 14 сущностями, описание API с DTO классами и контроллерами, результаты тестирования (85 юнит-тестов), три сценария нагрузочного тестирования JMeter, 18 макетов интерфейса и 18 скриншотов реализованного приложения.

Система реализована на Java с использованием Spring Framework, H2 Database для хранения данных и React для фронтенда. Все компоненты протестированы и готовы к развёртыванию.

Пояснительная записка содержит 24 страниц, 0 рисунков, 1 таблиц.

ABSTRACT

The explanatory note contains a description of the design and development of the «Re-Center Resort» information system — a distributed web system for automating business processes of a tourist complex.

The document presents: analysis of the subject area, description of five main use cases, functional and non-functional requirements, system architecture, data model with 14 entities, API description with DTO classes and controllers, testing results (85 unit tests), three JMeter load testing scenarios, 18 interface mockups and 18 screenshots of the implemented application.

The system is implemented in Java using Spring Framework, H2 Database for data storage and React for the frontend. All components are tested and ready for deployment.

The explanatory note contains 24 pages, 0 images, 1 tables.

					ВЛГУ.09.03.02.17.00.00						
Изм.	Лист	№ докум.	Подп.	Дата							
Разраб.		И. И. Иванов			к курсовому проекту по дисциплине «Распределённые программные системы» на тему			Лит.	Лист	Листов	
Пров.		И. И. Иванов								2	24
								АБВ-123			
Н. контр.											
Утв.											

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	4
ОПИСАНИЕ ПРЕДМЕТНОЙ ОБЛАСТИ	4
Основные проблемы, решаемые системой	5
Категории пользователей	5
АНАЛИЗ БИЗНЕС-ПРОЦЕССА	5
Описание прецедентов	5
Диаграмма прецедентов	9
ТРЕБОВАНИЯ К СИСТЕМЕ	9
Функциональные требования	9
Нефункциональные требования	10
МАКЕТ ИНТЕРФЕЙСА	10
Общая структура интерфейса	10
DTO (Data Transfer Objects)	11
Контроллеры	14
ТЕСТИРОВАНИЕ	18
Юнит-тесты	18
Сценарии нагрузочного тестирования (JMeter)	18
РЕАЛИЗАЦИЯ ИНТЕРФЕЙСА	20
Страницы аутентификации	21
Основные страницы клиента	21
Административная панель	22
ЗАКЛЮЧЕНИЕ	23
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	24

					ВЛГУ.09.03.02.17.00.00			
Изм.	Лист	№ докум.	Подп.	Дата				
Разраб.		И. И. Иванов			к курсовому проекту по дисциплине «Распределённые программные системы» на тему			
Пров.		И. И. Иванов						
Н. контр.								
Утв.								
						Лит.	Лист	Листов
							3	24
					АБВ-123			

ВВЕДЕНИЕ

Развитие информационных технологий и интернета привело к необходимости создания современных распределённых систем, которые обеспечивают масштабируемость, надёжность и удобство использования. Туристический и рекреационный бизнес не является исключением — базы отдыха и курортные комплексы нуждаются в автоматизации своих бизнес-процессов для повышения эффективности и улучшения качества обслуживания клиентов.

Данный курсовой проект посвящен проектированию и разработке информационной системы «База отдыха Re-Center», которая предназначена для автоматизации ключевых процессов: бронирования услуг, управления клиентами, обработки платежей и публикации актуальной информации. Система разработана с использованием современных технологий и архитектурных подходов, обеспечивающих высокую производительность, безопасность и удобство использования [1].

Целью проекта является создание полнофункциональной веб-системы, которая позволит базе отдыха эффективно управлять своей деятельностью, предоставляя клиентам удобный интерфейс для бронирования услуг и управления своими заказами. Система должна поддерживать различные роли пользователей: администраторов, менеджеров, зарегистрированных клиентов и гостей.

В пояснительной записке описаны все этапы разработки системы: анализ предметной области, проектирование архитектуры, реализация компонентов, тестирование и развёртывание. Документ содержит диаграммы, макеты интерфейса, описание API, результаты тестирования и скриншоты реализованного приложения.

ОПИСАНИЕ ПРЕДМЕТНОЙ ОБЛАСТИ

База отдыха «Re-Center» представляет собой комплекс услуг для отдыха и досуга клиентов, включающий проживание в коттеджах, спортивные мероприятия, экскурсии, аренду оборудования и развлекательные программы. В современном формате работы базы отдыха сталкиваются с необходимостью автоматизации ключевых процессов: бронирования услуг, учета клиентов, управления платежами и публикации актуальной информации.

Основные проблемы, решаемые системой

- Разрозненность информации об услугах и ценах
- Отсутствие удобной системы онлайн-бронирования
- Ручное распределение участников по возрастным группам
- Отсутствие механизма предоплаты и онлайн-оплаты
- Неэффективная система отмены бронирований
- Отсутствие системы мотивации клиентов
- Недостаток актуальных новостей и объявлений

Категории пользователей

Система предназначена для четырех основных категорий пользователей:

- Администраторы — управление услугами, бронированиями, новостями и статистикой
- Менеджеры — управление бронированиями и клиентами
- Зарегистрированные клиенты — просмотр услуг, бронирование, управление своими заказами
- Гости (незарегистрированные пользователи) — просмотр услуг и новостей

АНАЛИЗ БИЗНЕС-ПРОЦЕССА

Описание прецедентов

Регистрироваться в системе

Предусловие: Пользователь (Гость) находится на главной странице веб-приложения базы отдыха.

Действующее лицо: Гость

Основной поток:

- 1) Гость нажимает кнопку «Регистрация» на главной странице
- 2) Система отображает форму регистрации с полями: имя, электронная почта, пароль
- 3) Гость заполняет обязательные поля и нажимает кнопку «Согласиться с условиями»

					ВЛГУ.09.03.02.17.00.00	Лист
Изм.	Лист	№ докум.	Подп.	Дата		5

- 4) Гость нажимает кнопку «Зарегистрироваться»
- 5) Система проверяет корректность данных и уникальность электронной почты
- 6) Система создаёт новую учётную запись Пользователя, присваивает ему статус «Клиент» и автоматически выполняет вход в систему
- 7) Система перенаправляет Клиента в его личный кабинет

Альтернативный поток: Электронная почта уже зарегистрирована

- На шаге 5 система обнаруживает, что введенная электронная почта уже существует в базе данных
- Система отображает сообщение об ошибке «Пользователь с таким email уже зарегистрирован» и предлагает восстановить пароль
- Поток прерывается

Постусловие: В системе создана новая учётная запись, пользователь аутентифицирован и получил роль «Клиент».

Бронировать услугу

Предусловие: Пользователь авторизован в системе как Клиент.

Действующее лицо: Клиент

Основной поток:

- 1) Клиент просматривает каталог услуг и нажимает кнопку «Подробнее» у выбранной услуги
- 2) Система отображает детальную информацию об услуге (описание, фотографии, цена)
- 3) Клиент выбирает дату заезда и дату выезда в соответствующих полях формы
- 4) Система проверяет доступность услуги на выбранные даты
- 5) Система рассчитывает итоговую стоимость, применяя доступные скидки
- 6) Клиент нажимает кнопку «Забронировать»
- 7) Система создаёт новое бронирование со статусом «Ожидание оплаты»
- 8) Система перенаправляет Клиента на страницу оплаты

					ВнГЧ.09.03.02.17.00.00	Лист
						6
Изм.	Лист	№ докум.	Подп.	Дата		

Альтернативный поток: Услуга недоступна на выбранные даты

- На шаге 4 система определяет, что услуга занята на запрашиваемый период
- Система отображает сообщение «Выбранные даты недоступны» и предлагает выбрать другие даты
- Поток возвращается к шагу 3

Постусловие: В системе создана новая заявка на бронирование, которая ожидает подтверждения оплаты.

Отменять бронирование

Предусловие: Клиент авторизован в системе и имеет активные бронирования.

Действующее лицо: Клиент

Основной поток:

- 1) Клиент открывает раздел «Мои бронирования» в личном кабинете
- 2) Система отображает список активных бронирований Клиента
- 3) Клиент нажимает кнопку «Отменить» у выбранного бронирования
- 4) Система запрашивает подтверждение действия
- 5) Клиент подтверждает отмену
- 6) Система проверяет временные условия отмены (срок до начала услуги)
- 7) Система рассчитывает сумму возврата средств согласно правилам базы отдыха
- 8) Система изменяет статус бронирования на «Отменено» и, если применимо, инициирует процесс возврата денежных средств
- 9) Система уведомляет Клиента об успешной отмене бронирования

Альтернативный поток: Нарушены правила отмены

- На шаге 6 система определяет, что срок для бесплатной отмены истёк
- Система отображает предупреждение о том, что будет удержан штраф, и запрашивает подтверждение отмены
- Поток возвращается к шагу 5

					ВлГЧ.09.03.02.17.00.00	Лист
						7
Изм.	Лист	№ докум.	Подп.	Дата		

Постусловие: Статус бронирования изменён на «Отменено», клиенту возвращена часть средств согласно политике отмены.

Добавлять новую услугу

Предусловие: Пользователь авторизован в системе как Администратор.

Действующее лицо: Администратор

Основной поток:

- 1) Администратор переходит в «Панель администратора» и выбирает раздел «Управление услугами»
- 2) Система отображает интерфейс управления услугами со списком существующих
- 3) Администратор нажимает кнопку «Добавить услугу»
- 4) Система отображает форму для ввода данных о новой услуге
- 5) Администратор заполняет обязательные поля: название, описание, категория, базовая цена, вместимость
- 6) Администратор нажимает кнопку «Сохранить»
- 7) Система проверяет корректность данных и сохраняет новую услугу в каталоге
- 8) Система уведомляет Администратора об успешном добавлении услуги

Альтернативный поток: Обнаружены ошибки в данных

- На шаге 7 система проверяет введенные данные и обнаруживает ошибки (например, не заполнено обязательное поле)
- Система отображает сообщение с указанием допущенных ошибок
- Поток возвращается к шагу 5 для исправления данных

Постусловие: В каталоге услуг системы создана новая позиция, которая стала доступна для бронирования Клиентами.

Публиковать новость

Предусловие: Пользователь авторизован в системе как Администратор.

Действующее лицо: Администратор

Основной поток:

					ВЛГУ.09.03.02.17.00.00	Лист
Изм.	Лист	№ докум.	Подп.	Дата		8

- 1) Администратор переходит в «Панель администратора» и выбирает раздел «Управление новостями»
- 2) Система отображает интерфейс управления новостями со списком существующих
- 3) Администратор нажимает кнопку «Добавить новость»
- 4) Система отображает форму для создания новости
- 5) Администратор заполняет поля: заголовок, содержание, дата публикации
- 6) Администратор нажимает кнопку «Опубликовать»
- 7) Система сохраняет новость и делает её видимой для всех посетителей сайта в разделе «Новости»
- 8) Система уведомляет Администратора об успешной публикации

Альтернативный поток: Сохранение черновика

- На шаге 6 Администратор нажимает кнопку «Сохранить черновик»
- Система сохраняет новость со статусом «Черновик», не публикуя её на сайте
- Поток завершается

Постусловие: Новая запись добавлена в блок новостей на главной странице и в разделе «Новости», где её могут видеть все посетители сайта.

Диаграмма прецедентов

ТРЕБОВАНИЯ К СИСТЕМЕ

Функциональные требования

Для клиентов

- Просмотр услуг и цен
- Онлайн-бронирование с выбором дат
- Личный кабинет с моими бронированиями
- Отмена бронирований

					ВЛГУ.09.03.02.17.00.00	Лист
Изм.	Лист	№ докум.	Подп.	Дата		9

Для администраторов

- Добавление и редактирование услуг
- Управление бронированиями
- Публикация новостей и акций
- Просмотр статистики

Системные функции

- Автоматические скидки (групповые, именинникам)
- Проверка доступности номеров
- Учет оплаты и возвратов

Нефункциональные требования

Производительность

- Быстрая загрузка страниц
- Стабильная работа при наплыве посетителей

Безопасность

- Защита данных клиентов
- Надежное хранение паролей

Удобство

- Простой и понятный интерфейс
- Работа на компьютерах
- Легкий процесс бронирования

МАКЕТ ИНТЕРФЕЙСА

Общая структура интерфейса

Верхняя навигационная панель содержит логотип «Re-Center», основные разделы сайта (Главная, Услуги, Новости, Контакты) и кнопки авторизации для пользователей. Подвал дублирует основную навигацию с добавлением копирайта «Владимир 2025» и обеспечивает постоянный доступ к ключевым разделам с любой страницы.

					ВлГУ.09.03.02.17.00.00	Лист
Изм.	Лист	№ докум.	Подп.	Дата		10

Модальное окно входа/регистрации позволяет пользователю войти или зарегистрироваться в веб-приложении.

Макет 1: Страница входа

DTO (Data Transfer Objects)

DTO — классы, необходимые для передачи данных между слоями приложения (например, между сервером и клиентом). Они изолируют структуру БД от внешних API и обеспечивают безопасность (например, не передают пароль). Содержат только необходимые поля и не включают бизнес-логику.

Аутентификация: AuthRequest и AuthResponse

Используются для аутентификации пользователя. AuthRequest содержит email и пароль, AuthResponse возвращает JWT-токен и данные пользователя [6].

```
@Data
public class AuthRequest {
    @NotBlank
    @Email
    private String email;

    @NotBlank
    private String password;
}

@Data
@NoArgsConstructor
@AllArgsConstructor
public class AuthResponse {
    private String token;
    private String type = "Bearer";
    private Long id;
    private String email;
    private String role;
}
```

Листинг 1. DTO для аутентификации

Регистрация: RegisterRequest

Содержит данные для регистрации нового пользователя с валидацией email, пароля и телефона.

```
@Data
public class RegisterRequest {
    @NotBlank
```

									Лист
Изм.	Лист	№ докум.	Подп.	Дата					11

```

    @Email
    private String email;

    @NotBlank
    @Size(min = 6, message = "Пароль должен быть не менее 6 символов")
    private String password;

    private String firstName;
    private String lastName;
    private String middleName;

    @Pattern(regex = "^\\+[1-9]\\d{1,14}$", message = "Неверный
формат телефона")
    private String phoneNumber;
}

```

Листинг 2. DTO для регистрации пользователя

Бронирование: BookingRequest и BookingResponse

BookingRequest используется для создания бронирования с проверкой дат. BookingResponse возвращает полную информацию о бронировании, включая статус оплаты.

```

@Data
public class BookingRequest {
    private Long serviceId;

    @Future(message = "Дата заезда должна быть в будущем")
    private LocalDateTime startDate;

    @Future(message = "Дата выезда должна быть в будущем")
    private LocalDateTime endDate;

    private Integer peopleCount;
    private BookingStatus status;
}

```

```

@Data
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class BookingResponse {
    private Long id;
    private Long serviceId;
    private String serviceTitle;
    private Long userId;
    private String userEmail;
    private LocalDateTime startDate;
    private LocalDateTime endDate;
}

```

									Лист
Изм.	Лист	№ докум.	Подп.	Дата					12


```

@Data
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class PaymentResponse {
    private Long id;
    private Long bookingId;
    private BigDecimal amount;
    private LocalDateTime paymentDate;
    private String status;
    private String paymentMethod;
}

```

Листинг 5. DTO для платежей

Контроллеры

AuthController — управление аутентификацией

Контроллер аутентификации и авторизации предоставляет endpoints для входа, регистрации и получения информации о текущем пользователе. Использует JWT-токены для защиты ресурсов.

```

@RestController
@RequestMapping("/api/auth")
public class AuthController {

    @PostMapping("/login")
    public ResponseEntity<?> authenticateUser(@Valid @RequestBody
AuthRequest loginRequest) {
        Authentication authentication =
authenticationManager.authenticate(
            new UsernamePasswordAuthenticationToken(
                loginRequest.getEmail(),
                loginRequest.getPassword()
            )
        );
        String jwt = jwtUtils.generateJwtToken(authentication);
        UserDetailsImpl userDetails = (UserDetailsImpl)
authentication.getPrincipal();

        AuthResponse response = new AuthResponse(
            jwt,
            userDetails.getId(),
            userDetails.getUsername(),

            userDetails.getAuthorities().iterator().next().getAuthority()
        );
        return ResponseEntity.ok(response);
    }
}

```

									Лист
Изм.	Лист	№ докум.	Подп.	Дата					14

```

    @PostMapping("/register")
    public ResponseEntity<?> registerUser(@Valid @RequestBody
RegisterRequest registerRequest) {
        if (userService.existsByEmail(registerRequest.getEmail())) {
            return ResponseEntity.badRequest().body("Email already in
use");
        }
        User user = User.builder()
            .email(registerRequest.getEmail())
            .passwordHash(encoder.encode(registerRequest.getPassword()))
            .firstName(registerRequest.getFirstName())
            .lastName(registerRequest.getLastName())
            .role(UserRole.CLIENT)
            .build();
        userService.create(user);
        return ResponseEntity.ok("User registered successfully");
    }

    @GetMapping("/me")
    public ResponseEntity<?> getCurrentUser(Authentication
authentication) {
        if (authentication == null) {
            return
ResponseEntity.status(HttpStatus.UNAUTHORIZED).body("Unauthorized");
        }
        UserDetailsImpl userDetails = (UserDetailsImpl)
authentication.getPrincipal();
        return userService.getById(userDetails.getId())
            .map(user -> ResponseEntity.ok(new UserResponse(...)))
            .orElse(ResponseEntity.status(HttpStatus.UNAUTHORIZED).body("Unauthorpr.
    }
}

```

Листинг 6. AuthController — endpoints для входа, регистрации и получения данных пользователя

BookingController — управление бронированиями

Контроллер бронирований обрабатывает создание, получение, обновление и удаление бронирований. Проверяет доступность услуги на выбранные даты и статус оплаты.

```

@RestController
@RequestMapping("/api/bookings")
public class BookingController {

    @PostMapping
    public ResponseEntity<?> createBooking(@Valid @RequestBody
BookingRequest request) {

```

						ВлГУ.09.03.02.17.00.00	Лист
Изм.	Лист	№ докум.	Подп.	Дата			15

```

        // Проверка доступности услуги на выбранные даты
        boolean isBusy = bookingService.getAll().stream()
            .filter(b ->
b.getService().getId().equals(request.getServiceId()))
            .anyMatch(b -> !
(b.getEndDate().isBefore(request.getStartDate())
||
b.getStartDate().isAfter(request.getEndDate())));

        if (isBusy) {
            return ResponseEntity.badRequest().body("Selected service
is already booked");
        }

        Booking booking = new Booking();
        booking.setService(service);
        booking.setUser(user);
        booking.setStartDate(request.getStartDate());
        booking.setEndDate(request.getEndDate());
        booking.setStatus(BookingStatus.PENDING);

        Booking createdBooking = bookingService.create(booking);
        return ResponseEntity.ok(toResponse(createdBooking));
    }

    @GetMapping("/my")
    public List<BookingResponse> getMyBookings(
        @RequestParam(name = "dateFrom", required = false)
LocalDate dateFrom,
        @RequestParam(name = "dateTo", required = false) LocalDate
dateTo,
        @RequestParam(name = "paid", required = false) Boolean
paid) {
        // Получение бронирований текущего пользователя с фильтрацией
        List<Booking> bookings =
bookingService.getFiltered(user.getId(), dateFrom, dateTo, paid);
        return bookings.stream()
            .map(this::toResponse)
            .toList();
    }

    @DeleteMapping("/{id}")
    public ResponseEntity<String> delete(@PathVariable Long id) {
        bookingService.delete(id);
        return ResponseEntity.ok("Booking deleted successfully");
    }
}

```

Листинг 7. BookingController — endpoints для создания, получения и удаления бронирований

					ВЛГУ.09.03.02.17.00.00	Лист
Изм.	Лист	№ докум.	Подп.	Дата		16

ServiceController — управление услугами

Контроллер услуг предоставляет endpoints для получения каталога услуг и управления ими (для администраторов). Поддерживает фильтрацию по категориям.

```
@RestController
@RequestMapping("/api/services")
public class ServiceController {

    @PostMapping
    @PreAuthorize("hasRole('ADMIN') or hasRole('MANAGER')")
    public ResponseEntity<ServiceResponse> create(@Valid @RequestBody
ServiceRequest request) {
        Category category =
categoryService.getById(request.getCategoryId())
                .orElseThrow(() -> new RuntimeException("Category not
found"));

        Service service = Service.builder()
                .title(request.getTitle())
                .description(request.getDescription())
                .price(request.getPrice())
                .maxPeople(request.getMaxPeople())
                .category(category)
                .build();

        Service created = serviceService.create(service);
        return
ResponseEntity.ok(EntityDtoMapper.toServiceResponse(created));
    }

    @GetMapping
    public ResponseEntity<List<ServiceResponse>> getAll() {
        List<ServiceResponse> responses =
serviceService.getAll().stream()
                .map(EntityDtoMapper::toServiceResponse)
                .toList();

        return ResponseEntity.ok(responses);
    }

    @GetMapping("/category/{categoryId}")
    public ResponseEntity<List<ServiceResponse>>
getByCategory(@PathVariable("categoryId") Long categoryId) {
        List<ServiceResponse> responses =
serviceService.getByCategory(category).stream()
                .map(EntityDtoMapper::toServiceResponse)
                .toList();

        return ResponseEntity.ok(responses);
    }
}
```

							ВлГУ.09.03.02.17.00.00	Лист
Изм.	Лист	№ докум.	Подп.	Дата				17

```

    }
}

```

Листинг 8. ServiceController — endpoints для управления услугами и фильтрации по категориям

ТЕСТИРОВАНИЕ

Юнит-тесты

Система содержит полный набор юнит-тестов для всех контроллеров и сервисов. Тесты проверяют корректность бизнес-логики, валидацию данных и обработку ошибок [5].

Тестовый класс	Количество тестов	Статус
ActivityControllerTest	6	Пройден
AuthControllerTest	5	Пройден
BookingControllerTest	8	Пройден
CategoryControllerTest	7	Пройден
DiscountControllerTest	8	Пройден
NewsControllerTest	8	Пройден
NotificationControllerTest	9	Пройден
PaymentControllerTest	7	Пройден
PromotionControllerTest	5	Пройден
ReviewControllerTest	8	Пройден
ServiceControllerTest	7	Пройден
UserControllerTest	7	Пройден

Таблица 1. Результаты юнит-тестирования

Итоговый результат: Всего запущено 85 тестов, все пройдены успешно (0 ошибок, 0 пропусков). Время выполнения: 9.790 сек.

Сценарии нагрузочного тестирования (JMeter)

Сценарий 1: Тестирование потока бронирования

Название: Re-Center Booking Flow Load Test

Описание: Нагрузочное тестирование аутентифицированного потока бронирования. Сценарий включает: вход в систему, просмотр услуг, создание бронирования, просмотр бронирования и удаление бронирования.

Параметры:

- Количество потоков (пользователей): 5
- Время разогрева: 30 сек
- Количество итераций: 10
- Время ожидания между запросами: 2000 мс
- Таймаут соединения: 10000 мс

Основные шаги:

- 1) Аутентификация пользователя (POST /api/auth/login)
- 2) Получение информации текущего пользователя (GET /api/auth/me)
- 3) Получение списка услуг (GET /api/services)
- 4) Создание бронирования (POST /api/bookings)
- 5) Получение моих бронирований (GET /api/bookings/my)
- 6) Удаление бронирования (DELETE /api/bookings/{id})

Результаты тестирования:

Сценарий 2: Тестирование административного потока управления услугами

Название: Re-Center Service Admin Flow Load Test

Описание: Нагрузочное тестирование административного потока управления услугами. Сценарий проверяет: вход администратора, создание категории, создание услуги, обновление услуги и удаление сущностей каталога.

Параметры:

- Количество потоков (администраторов): 5
- Время разогрева: 30 сек
- Количество итераций: 10
- Время ожидания между запросами: 2000 мс
- Таймаут соединения: 10000 мс

Основные шаги:

- 1) Аутентификация администратора (POST /api/auth/login)
- 2) Проверка роли ADMIN в ответе
- 3) Получение информации текущего пользователя (GET /api/auth/me)

									Лист
									19
Изм.	Лист	№ докум.	Подп.	Дата					

- 4) Создание категории услуг (POST /api/categories)
- 5) Создание новой услуги (POST /api/services)
- 6) Обновление услуги (PUT /api/services/{id})
- 7) Удаление услуги (DELETE /api/services/{id})

Результаты тестирования:

Сценарий 3: Тестирование потока профиля пользователя

Название: Re-Center Profile Flow Load Test

Описание: Нагрузочное тестирование аутентифицированного потока профиля. Сценарий включает: вход в систему, получение информации профиля, обновление профиля и просмотр журнала активности.

Параметры:

- Количество потоков (пользователей): 5
- Время разогрева: 30 сек
- Количество итераций: 10
- Время ожидания между запросами: 2000 мс
- Таймаут соединения: 10000 мс

Основные шаги:

- 1) Аутентификация пользователя (POST /api/auth/login)
- 2) Получение информации текущего пользователя (GET /api/auth/me)
- 3) Получение профиля пользователя (GET /api/users/{id})
- 4) Обновление профиля (PUT /api/users/{id})
- 5) Получение журнала активности (GET /api/activities)

Результаты тестирования:

РЕАЛИЗАЦИЯ ИНТЕРФЕЙСА

Фронтенд-приложение разработано на React с использованием JavaScript и современных инструментов разработки [7]. Интерфейс обеспечивает удобное взаимодействие для всех категорий пользователей: гостей, клиентов и администраторов.

									Лист
Изм.	Лист	№ докум.	Подп.	Дата					20

Страницы аутентификации

Страница входа

Позволяет пользователям войти в систему с использованием email и пароля. При успешной аутентификации выдается JWT-токен.

Страница регистрации

Форма регистрации новых пользователей с валидацией email, пароля и контактной информации. После регистрации пользователь автоматически получает роль CLIENT.

Основные страницы клиента

Каталог услуг

Отображает полный список доступных услуг с фильтрацией по категориям, ценой и другим параметрам. Каждая услуга представлена карточкой с фотографией, описанием и кнопкой «Подробнее».

Детали услуги

Детальная страница услуги с полной информацией, фотографиями, характеристиками и блоком бронирования. Позволяет выбрать даты и количество участников.

Акции и скидки

Раздел с актуальными акциями, скидками и специальными предложениями. Отображает условия применения скидок и сроки действия.

Новости

Блок новостей и объявлений базы отдыха. Содержит информацию о событиях, обновлениях и важных сообщениях.

Мои бронирования

Личный кабинет клиента с таблицей активных и завершенных бронирований. Позволяет просмотреть детали, отменить бронирование или оставить отзыв.

Уведомления

Центр уведомлений с информацией о статусе бронирований, платежах, новых акциях и системных сообщениях.

Профиль пользователя

Страница профиля с личной информацией, контактными данными и историей активности. Позволяет редактировать данные и просматривать статистику.

Административная панель

Общий экран управления

Дашборд администратора с ключевыми метриками: количество новых бронирований, выручка, загрузка услуг и статистика пользователей.

Управление бронированиями

Интерфейс для просмотра, редактирования и управления всеми бронированиями в системе. Позволяет изменять статусы и отслеживать платежи.

Управление услугами

Интерфейс для добавления, редактирования и удаления услуг. Позволяет управлять категориями, ценами, описаниями и фотографиями.

Управление новостями

Интерфейс для создания, редактирования и публикации новостей. Поддерживает форматирование текста и загрузку изображений.

Управление акциями

Интерфейс для создания и управления акциями и скидками. Позволяет применять скидки к услугам или категориям с указанием сроков действия.

Управление скидками

Интерфейс для управления персональными скидками пользователей. Позволяет назначать скидки по различным критериям.

					ВЛГУ.09.03.02.17.00.00	Лист
Изм.	Лист	№ докум.	Подп.	Дата		22

Модерация отзывов

Интерфейс для просмотра и модерации отзывов пользователей. Позволяет одобрять, отклонять или удалять отзывы.

Управление платежами

Интерфейс для отслеживания платежей, проверки статусов и управления возвратами. Показывает историю всех транзакций.

Страница ошибки 404

Страница, отображаемая при попытке доступа к несуществующему ресурсу. Содержит информацию об ошибке и ссылку на главную страницу.

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсового проекта была разработана полнофункциональная информационная система «База отдыха Re-Center», которая решает основные проблемы автоматизации бизнес-процессов туристического комплекса.

Основные достижения проекта:

- Спроектирована архитектура системы с использованием паттернов REST API и многоуровневой архитектуры
- Разработана реляционная база данных с 14 основными сущностями и правильно организованными связями
- Реализован полный набор API endpoints для управления услугами, бронированиями, платежами и контентом
- Создан удобный веб-интерфейс для всех категорий пользователей: гостей, клиентов и администраторов
- Реализована система аутентификации и авторизации с использованием JWT-токенов
- Разработана система управления скидками и акциями с автоматическим расчётом стоимости
- Внедрена система уведомлений для информирования пользователей о статусе их заказов

Качество разработанной системы подтверждается результатами тестирования:

					ВЛГУ.09.03.02.17.00.00	Лист
Изм.	Лист	№ докум.	Подп.	Дата		23

- Все 85 юнит-тестов пройдены успешно
- Нагрузочное тестирование показало стабильную работу системы при одновременном подключении 5 пользователей с 10 итерациями операций
- Система обеспечивает быструю загрузку страниц и отзывчивый интерфейс

Система готова к развёртыванию и использованию в реальной среде. Архитектура позволяет легко масштабировать приложение при увеличении нагрузки и добавлять новые функции без значительных изменений в существующем коде.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

- 1) H2 Database Engine: официальная документация. — URL: <https://www.h2database.com/html/main.html> (дата обращения: 15.10.2025).
- 2) Project Lombok: официальная документация. — URL: <https://projectlombok.org/> (дата обращения: 20.11.2025).
- 3) MapStruct: официальная документация. — 2026. — URL: <https://mapstruct.org/> (дата обращения: 10.12.2025).
- 4) Mockito: framework documentation. — 2026. — URL: <https://site.mockito.org/> (дата обращения: 15.01.2026).
- 5) Как упростить себе жизнь: удобные юнит-тесты с JUnit и AssertJ. — URL: <https://habr.com/ru/companies/skillbox/articles/767804/> (дата обращения: 20.02.2026).
- 6) Первые шаги в Spring Security с JWT. — 2025. — URL: https://habr.com/ru/companies/spring_aio/articles/909448/ (дата обращения: 15.03.2026).
- 7) Первое приложение на Spring Boot + ReactJS. — URL: <https://habr.com/ru/users/kadmser/> (дата обращения: 15.05.2026).